

华东师范大学数据科学与工程学院实验报告

课程名称：AI 基础

年级：2024 级

上机实践日期：

2026 年 3 月 22 日

指导教师：杨彬

姓名：唐一嘉

上机实践名称：实验3

学号：10245101434

一、实验任务

1. 算法回顾

图最短路问题，有测评机

- 1.1: BFS (每条边的权重为1)
- 1.2: 朴素版Dijkstra (图最短路)
- 1.3: 堆优化版Dijkstra (图最短路)

2. 八数码问题

经典搜索问题，有测评机

- 2.1: DFS (解存在性问题)
- 2.2: BFS (求解最少步数)
- 2.3: Dijkstra (特殊的A star)
- 2.4: A star (求解最少步数)

二、实验过程

1. 图最短路问题

1.1 BFS (shortest_path/bfs_shortest_path.py)

1.1.1 实现功能

- 在无权有向图中求 1 号点到 n 号点的最短边数。
- 不可达输出 -1。

1.1.2 核心思路

```
dist = [-1] * (n + 1)
dist[1] = 0

q = deque([1])

while q:
    u = q.popleft()

    # 一旦求出到 n 的距离, 直接退出
    if u == n:
        break

    for v in range(1, n + 1):
        if adj[u][v] == 1 and dist[v] == -1: # 若有边且未访问过
            dist[v] = dist[u] + 1
            q.append(v)
```

- 用邻接矩阵 `adj` 存图。
- BFS 从 1 号点分层扩展, `dist[v]` 保存最短层数。
- 当弹出节点为 `n` 时提前退出。

1.1.3 复杂度分析

- 由于使用的是邻接矩阵, 每次出队要扫描整行:
 - 时间复杂度: $O(n^2)$ 。
 - 空间复杂度: $O(n^2)$ 。

1.1.4 测试截图

```
(.venv) PS D:\code\AIBasics\Project1\shortest_path> python .\bfs_shortest_path.py
4 5
1 2
2 3
3 4
1 3
1 4
^Z
❖ 1
```

1.2 朴素版 Dijkstra (shortest_path/dijkstra_naive.py)

1.2.1 实现功能

- 在带权有向图 (非负权) 中求 1 到 `n` 的最短路。
- 不可达输出 `-1`。

1.2.2 核心思路

```
# 迭代n次，每次确定一个最短距离的点
for _ in range(n):
    t = -1
    # 在未确定最短路的点集中，寻找距离目前已知最短的一个点
    for j in range(1, n + 1):
        if not st[j] and (t == -1 or dist[j] < dist[t]):
            t = j

    # 所有点如果不连通了，可以提前结束
    if t == -1:
        break

    st[t] = True

    # 用新找到的距离起点的最短的点t更新其它所有点距离起点的最短距离
    for j in range(1, n + 1):
        if dist[t] + g[t][j] < dist[j]:
            dist[j] = dist[t] + g[t][j]
```

- 邻接矩阵存图，重边取最小权值。
- 每轮在线性扫描中选取当前未确定、`dist` 最小的点 `t`。
- 用 `t` 对所有点做松弛。

1.2.3 复杂度分析

- 仍然因为使用的是邻接矩阵，每次出队要扫描整行：
 - 时间复杂度： $O(n^2)$ 。
 - 空间复杂度： $O(n^2)$ 。

1.2.4 测试截图

```
(.venv) PS D:\code\AIBasics\Project1\shortest_path> python .\dijkstra_naive.py
3 3
1 2 2
2 3 1
1 3 4
^Z
3
```

1.3 堆优化版 Dijkstra (`shortest_path/dijkstra_heap.py`)

1.3.1 实现功能

- 同样求 1 到 n 的最短路，使用优先队列优化 Dijkstra。

1.3.2 核心思路

```
# 优先队列 (距离, 节点 id), 一开始 1 号点入队距离为 0
pq = [(0, 1)]

while pq:
    # 每次弹出当前距离 1 号点最近的点 u
    d, u = heapq.heappop(pq)

    # 堆中可能保留着更新前的冗余记录, 若已被确定最短路径, 则忽略
    if st[u]:
        continue

    st[u] = True

    # 优化: 一旦求出到 n 的最短路, 提前返回
    if u == n:
        break

    # 遍历 u 的所有邻接边 (只遍历实际存在的边)
    for v, w in g[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            heapq.heappush(pq, (dist[v], v))
```

- 维护最小堆 `pq=(dist,node)`, 每次弹出当前最短候选点。
- `st[u]` 标记最短路已确定点, 跳过重复过期记录。
- 发现更短路径时更新 `dist[v]` 并入堆。
- 弹出 `n` 后提前结束。

1.3.3 复杂度分析

- 时间复杂度: $O(m \log n)$
 - 建图: $O(m)$ - 读入 m 条边到邻接表
 - 堆操作: 每条边最多入队 1 次, 每次堆操作 $O(\log n)$
 - 总操作数: $m \text{ 条边} \times O(\log n) = O(m \log n)$
- 空间复杂度: $O(n + m)$
 - 邻接表: $O(m)$ - 存储 m 条边
 - 距离数组: $O(n)$
 - 标记数组: $O(n)$
 - 优先队列: 最坏 $O(m)$
 - 总计: $O(n + m)$

1.3.4 测试截图

```
(.venv) PS D:\code\AIBasics\Project1\shortest_path> python .\dijkstra_heap.py
3 3
1 2 2
2 3 1
1 3 4
^Z
3
```

2. 八数码问题

注: V (Vertex): 状态总数 $= 9!/2 = 181,440$ 个可达状态

E (Edge): 状态转移的总边数 $\approx V \times \text{平均分支因子} \approx 181,440 \times 2.67 \approx 484,445$ 条边

2.1 DFS (eight_puzzle/eight_puzzle_dfs.py)

2.1.1 实现功能

- 判断 3×3 八数码状态是否有解。
- 输入 9 个字符 (1~8 与 x)，输出 1 (有解) 或 0 (无解)。

2.1.2 核心思路

```
def dfs_can_reach(state: str, visited: set) -> bool:
    # 找到目标状态, 返回 True
    if state == TARGET:
        return True

    # 标记当前状态为已访问
    visited.add(state)

    x_idx = state.index('x')
    state_list = list(state)

    # 尝试所有可能的移动
    for nxt in NEIGHBORS[x_idx]:
        # 交换 x 和相邻位置
        state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]
        next_state = ''.join(state_list)
        # 恢复状态, 继续下一次交换
        state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]

        # 如果未访问过该状态, 继续 DFS
        if next_state not in visited:
            if dfs_can_reach(next_state, visited):
                return True
```

- 尝试所有可能的移动
- 直到找到目标状态, 返回 True

2.1.3 复杂度分析

- 时间复杂度: $O(V + E) = O(\text{状态数} \times \text{转移数})$
- 空间复杂度: $O(V) = O(\text{状态数})$

2.1.4 测试截图

```
(.venv) PS D:\code\AIBasics\Project1\eight_puzzle> python .\eight_puzzle_solvable.py
2 3 4 1 5 x 7 6 8
^Z
1
```

2.2 BFS (eight_puzzle/eight_puzzle_bfs.py)

2.2.1 实现功能

- 求八数码从初始状态到目标状态 12345678x 的最少步数。
- 无解输出 -1。

2.2.2 核心思路

```
def bfs_min_steps(start: str) -> int:
    if start == TARGET:
        return 0

    queue = deque([start])
    dist = {start: 0}

    while queue:
        state = queue.popleft()
        step = dist[state]
        x_idx = state.index('x')

        state_list = list(state)
        for nxt in NEIGHBORS[x_idx]:
            state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]
            next_state = ''.join(state_list)
            state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]

            if next_state in dist:
                continue

            if next_state == TARGET:
                return step + 1

            dist[next_state] = step + 1
            queue.append(next_state)

    return -1
```

- 使用 BFS（广度优先搜索）按层扩展状态图。
- `dist[state]` 记录首次到达该状态的步数；BFS 首次到达即为最短步数。
- 用 `NEIGHBORS` 预定义每个位置的可交换下标，减少边界判断开销。

2.2.3 复杂度分析

- 状态总数上界为 $9! = 362880$ ，可达状态约一半。
- 时间复杂度： $O(|V|+|E|)$ 。
- 空间复杂度： $O(|V|)$ （队列 + dist 表）。

2.2.4 测试截图

```
(.venv) PS D:\code\AIBasics\Project1\eight_puzzle> python .\eight_puzzle_bfs.py
2 3 4 1 5 x 7 6 8
^Z
19
```

2.3 Dijkstra (eight_puzzle/eight_puzzle_dijkstra.py)

2.3.1 实现功能

- 求八数码从初始状态到目标状态 `12345678x` 的最少步数。
- 无解输出 `-1`。

2.3.2 核心思路

```
while pq:
    d, state = heapq.heappop(pq)

    # 如果当前距离大于已知的最短距离, 跳过 (冗余数据)
    if d > dist.get(state, float('inf')):
        continue

    # 找到目标状态, 返回距离
    if state == TARGET:
        return d

    x_idx = state.index('x')
    state_list = list(state)

    # 遍历所有可能的移动
    for nxt in NEIGHBORS[x_idx]:
        # 交换 x 和相邻位置
        state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]
        next_state = ''.join(state_list)
        # 恢复状态, 继续下一次交换
        state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]

        # 新的距离 (每次移动权重为 1)
        new_dist = d + 1

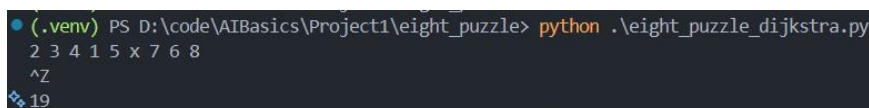
        # 如果找到更短的路径, 更新
        if new_dist < dist.get(next_state, float('inf')):
            dist[next_state] = new_dist
            heapq.heappush(pq, (new_dist, next_state))
```

- 由于本题的图是无权图, 因此 Dijkstra 退化为 BFS
- 由于所有边权重相等, Dijkstra 的优先队列实际上也是按层处理

2.3.3 复杂度分析

- 时间复杂度: $O((V + E) \log V)$ (多了堆的 $\log$ 因子)
- 空间复杂度: $O(|V|)$

2.3.4 测试截图



```
(.venv) PS D:\code\AIBasics\Project1\eight_puzzle> python .\eight_puzzle_dijkstra.py
2 3 4 1 5 x 7 6 8
^Z
19
```

2.4 A star (eight_puzzle/eight_puzzle_astar.py)

2.4.1 实现功能

- 求八数码从起点到目标状态的操作序列 (u/d/l/r)。
- 无解输出 unsolvable。

2.4.2 核心思路

```

while heap:
    f_val, g_val, state = heapq.heappop(heap)

    # 跳过过期状态
    if g_val != g_score.get(state, -1):
        continue

    if state == TARGET:
        return reconstruct_path(parent, state)

    x_idx = state.index("x")
    x_row, x_col = divmod(x_idx, 3)
    state_list = list(state)

    for dr, dc, action in MOVES:
        nr, nc = x_row + dr, x_col + dc
        if nr < 0 or nr >= 3 or nc < 0 or nc >= 3:
            continue

        nxt = nr * 3 + nc
        state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]
        next_state = "".join(state_list)
        state_list[x_idx], state_list[nxt] = state_list[nxt], state_list[x_idx]

        new_g = g_val + 1
        if new_g < g_score.get(next_state, 10**9):
            g_score[next_state] = new_g
            parent[next_state] = (state, action)
            heapq.heappush(heap, (new_g + manhattan(next_state), new_g, next_state))

return "unsolvable"

```

- 先用逆序对剪枝无解输入。
- 使用 A*：
 - $g(n)$: 起点到当前状态的步数；
 - $h(n)$: 曼哈顿距离启发函数；
 - $f(n)=g(n)+h(n)$ 。
- 用最小堆按 f 取下一个扩展状态。
- g_score 维护当前已知最优 g ，并通过“过期状态跳过”避免重复无效扩展。
- $parent[state]=(prev, action)$ 记录前驱和动作，命中目标后回溯重建路径。

2.4.3 复杂度分析

- 时间复杂度: $O((V + E) \log V)$
- 空间复杂度: $O(|V|)$

2.4.4 测试截图

```

(.venv) PS D:\code\AIBasics\Project1\eight_puzzle> python .\eight_puzzle_astar.py
2 3 4 1 5 x 7 6 8
^Z
u1l1ddrurdl1lurdruldr

```

三、总结

在本次实验中，我完成了一系列最短路径算法和八数码问题求解，实现了多种经典搜索算法。最短路径问题：实现了 BFS（无权图）、朴素 Dijkstra（稠密图）、堆优化 Dijkstra（稀疏图，邻接表实现）；八数码问题：实现了 DFS（解存在性判断）、BFS、Dijkstra、A*（曼哈顿距离启发）。

通过本次实验，我掌握了不同算法的差异，理解了数据结构选择的重要性（邻接矩阵 vs 邻接表）理解了启发式搜索的优势，A* 通过启发函数显著减少扩展节点数。